Tech Spec Template - Project Name

Author/s: Aditi Maheshwari
Team/Pod: SPINE Edge | BU: Platform
Published Date: Jun 24, 2022

Reviewer Name	Reviewed Date	Status
Venkat V		Approved
Shashank Kumar		Approved
Cell Arch Team		Not started
Data Hub Team		Not started
Analytics Team		Not started
i18N Team		Not started

Table of Contents

1. Problem Statement
2. Introduction & Scope
- 2.1. Tenets
- 2.2. Relevant Resources

3. Out of Scope
4. Futuristic Scope
5. Assumptions, Goals & Non-Goals
- 5.1. Assumptions
- 5.2. Goals
- 5.3. Non-Goals

6. Domain Design
7. Current Architecture / Current HLD
8. Final Approach - Specifications
- 8.1. List of Possible Solutions
  - 8.1.1. Event Modeling
  - 8.1.2. Domain Design
  - 8.1.3. APIs
- 8.2. Assumptions

8.3. Possibility of Open Sourcing
8.4. Possibility on Patent
8.5. Data model/Schema Changes
8.6. Business Logic Changes
8.7. Miscellaneous Questions
9. Non-Functional Requirements (NFRs)
9.1. Scalability
9.2. Availability
9.3. Security
9.4. Compliance
9.5. Reliability
9.6. Infra Cost
10. Agentic Development Spec - Agentic SDLC [Mandatory]
10.1. Agent Objective
10.2. Repositories in Scope
10.3. Detailed Implementation Spec
10.4. Success Criteria
10.5. Test Requirements
10.6. Constraints & Guardrails
10.7. Spec Location
11. Feature Dependencies & SLAs
11.1. Upstream Dependencies
11.2. Downstream Dependencies
12. Testing Plan
12.1. Test Cases
12.2. UAT Testing
12.3. Performance/Load testing
13. Go-live plan
13.1. Production Rollout and Ramp plan
13.2. Backward Compatibility
13.3. Rollback plan
14. Monitoring & Logging
15. Milestones & Timelines
16. Glossary
17. Appendix

💡 ***Note***

Read following docs to get expert on different topics -

- Format and Structure Your Doc
- How to document code in Google Doc
- API Design Guide
- Put Google Doc in Presentation Mode

Full Reading List - Link

Important: Agentic Development Spec (Section 10) is mandatory now and can be kept and maintained with Claude in Github but should be linked here in Section 10.

1. Problem Statement

Since the problem would already be deeply analyzed on Product end, just a brief (4-5 points) on the problem being solved by the above feature should be covered in this section. But an engineer knowing what problem is exactly being targeted, usually helps in coming up with the best approaches to solve.

2. Introduction & Scope

This section should contain a high level introduction of the feature, clearly indicating the scope of the feature and a summary of the executive/project that the feature is coming from in the PR format

2.1. Tenets

Here are the tenets unless you know better
Security is our top priority:

- We will strive to use existing Authentication and Authorization mechanisms/libraries
- We will not store credentials in the code
- We handle customer data carefully, e.g. We will not log PII data
- We will instrument

2.2. Relevant Resources

Link to Product Spec - Link
Link to Figma Design - Link
Link/Description around functional requirements

3. Out of Scope

Clearly call out the features/enhancements which would be out of scope as far as this document/goal is concerned. Mention them as either a future scope to the feature, but point it out upfront to keep expectations clear.

4. Futuristic Scope

You are expected to be like Christopher Nolan and see how this can evolve, in case you can :). If you can't ignore this section and move on. The above was a joke. Please use wisdom as appropriate.

5. Assumptions, Goals & Non-Goals

5.1. Assumptions

Add what assumptions you're making while coming up with the goals and non-goals

5.2. Goals

Add bulleted goals specific to the feature.

5.3. Non-Goals

Call out clearly the non-goals of the feature as well.
For instance, if the feature is important, we would be releasing it irrespective of the costs being incurred. Hence, it's not a goal but however, it's not out of scope.

6. Domain Design

- Domain Model
    - Descriptions of Key Entities, Value Objects, and Aggregates
    - Bounded Contexts and Their Relationships
    - Context map illustration
    - Description of boundaries and communications
    - Core, Supporting, and Generic Subdomains
- Glossary of Domain Terms (Ubiquitous Language)
- Key Business Rules and Invariants
- Event Modeling: Link

7. Current Architecture / Current HLD

Add the current flow/architecture of the system.

8. Final Approach - Specifications

This section will highlight which approach is being picked up and why

8.1. List of Possible Solutions

Add only the links of all the other approaches and detail out only the final approach.

The other approaches should elaborate on each of the below:

- Describes what will be changes being made to the codebase
- Diagrammatic representation of the process flow describing the stages
- Total effort estimation in days/sprints/months to be put here.
- Enter bulleted pros here for the approach
- Enter bulleted cons here for the approach

This section should detail out all possible approaches to resolve the problem statement defined above - including any diagrammatic, flowchart representations that might be required to explain the problem statement.
Each approach should conclude with a **pros and cons** section for that particular approach

8.1.1. Event Modeling

https://eventmodeling.org/posts/what-is-event-modeling/

8.1.2. Domain Design

8.1.3. APIs

8.2. Assumptions

Add only if applicable, mention any assumptions you've made in the proposed approach.. Some examples:

1. The SLA for this product/application should be bound by X
2. We need a specific kind of hardware for running on the approaches(e.g. Need GPU machines for running ML experiments)
3. Do we specifically need a tool/information etc as a mandate for running this product? For instance, i should have requisite analytical information for us to make judicious implementation.
4. The requisite cost for running this application is at the minimum X dollars.
5. I cannot run this project without asana as a tool (bad example, but read the subtext)
6. This solution will trade off around efficiency vs cost vs latency as this is a 0-1 product
7. This explicitly is meant to scale for a minimum of 10k tps

8.3. Possibility of Open Sourcing

Call out the possibility and process around open sourcing the feature and maintaining it.

8.4. Possibility on Patent

Check if the feature qualifies for being patented.

8.5. Data model/Schema Changes

Add only if applicable

- Schema definitions
- New data models
- Modified data models
- Data validation methods
- Data Sizing Considerations
- Data Archival/Partitioning Strategy
- Add any more relevant links as required.

8.6. Business Logic Changes

Add only if applicable

- API changes
- Pseudocode
- Flowcharts
- Error states
- Failure scenarios
- Conditions that lead to errors and failures
- Limitations

8.7. Miscellaneous Questions

Answer questions if only relevant, add more questions as required

- What are the limitations of the solution?
- How will it recover in the event of a failure?
- How will it cope with future requirements?

9. Non-Functional Requirements (NFRs)

Talk about each as applicable and call out the expectations/risks from the feature.

9.1. Scalability

9.2. Availability

9.3. Security

9.4. Compliance

9.5. Reliability

9.6. Infra Cost

10. Agentic Development Spec - Agentic SDLC [Mandatory]

This section is required when the implementation is intended to be executed by an AI agent (e.g. Claude) end-to-end - which is all cases now. It serves as the success criteria and execution blueprint for agentic development.

Till Section 9 is good for initial discussion and alignment between teams on what is going to be built, once that is closed we should draft 10 directly with Claude on Github and link it here.

Agentic Development Spec should be kept in Github in the main repo where we are developing as it is easier to create and maintain it there with Claude.

We have a first draft of the sub headers here - we would make it better as we learn.

Reference specs of trying out fully autonomous agentic SDLC (based on learning on these we would make Section 10 better):

1. Payments Service Agentic SDLC Spec: Arjun Tomer : PG Router Redesign
2. Magic Targeting+Segments Agentic SDLC Spec:
    a. Static: Monark Dedakiya Link
    b. Realtime: Alok Kumar Singh Link

10.1. Agent Objective

Define the high-level goal for the agent in plain language. What does "done" look like from the agent's perspective?

10.2. Repositories in Scope

List all repos the agent will need to read from or write to, with brief context on each.

Repo	Purpose	Branch Strategy

10.3. Detailed Implementation Spec

This is the core input to the agent. It must be granular enough that no ambiguity exists. Include:

- Precise file/module level changes expected in each repo
- API contracts, data models, and business logic to be implemented
- Sequence of operations across repos (what depends on what)
- Environment setup instructions for devstack

10.4. Success Criteria

Define what the agent must produce for the milestone to be considered complete:

- Both repos boot successfully in devstack
- All unit tests pass
- All slit tests pass
- All e2e tests pass against devstack
- Detailed spec saved to `specs/` folder

10.5. Test Requirements

Explicitly call out the test coverage the agent is expected to write:

Test Type	Scope	Location
Unit Tests	Per Service / module	<repo specific>
Slit Tests	Cross-service contracts	
E2E Tests	Full flow on devstack	

10.6. Constraints & Guardrails

- Coding standards and patterns the agent must follow @backend engineer

- Libraries/frameworks it must use or avoid
- Any human checkpoints required mid-execution

10.7. Spec Location

The agent must produce a detailed implementation spec at `specs/<spec-name>.md` **in Github Repo only** as its **first deliverable**, before writing any code.

11. Feature Dependencies & SLAs

Callout upstream and downstream applications that will get impacted

11.1. Upstream Dependencies

Add SLO/SLA dashboard links for existing services.

Sr. No.	Service	Impact	SLA (if applicable)	POC for service
1	abc			Owner/expert on abc

11.2. Downstream Dependencies

Note: i18N, Cell Arch, DataHub and Analytics are mandatory dependencies and must be analysed for each tech initiate.

Sr. No.	Service	Impact	SLA (if applicable)	POC for service
1	**i18N**			Rajeev Rai Shankar Parimi
2	**Cell Arch**			Rajeev Rai Nanda Kishore B…
3	**Data Hub**			Nitesh Jain Anuj Jain
4	**Analytics**			Abhinav Vajpayee

12. Testing Plan

Lay down test scenarios

12.1. Test Cases

(UTs, ITs, regression suites, UI/UX tests and manual test efforts)
Capturing the results with upstream and downstream applications and validating the results against expected outputs

Add links here only.

12.2. UAT Testing

Capturing User acceptance test results, validating against genuine data

12.3. Performance/Load testing

Capturing any load testing that we wish to perform

13. Go-live plan

13.1. Production Rollout and Ramp plan

Add what will be done to progressively roll out the feature in production. You can take cues from product spec also to see how we can support the ramp up whether in a phased manner, in one shot or through certain gating criterias as needed.

13.2. Backward Compatibility

13.3. Rollback plan

Add what will be done in case of an immediate rollback requirement and how it will be implemented.

14. Monitoring & Logging

Jot down all metrics that need to be monitored for the changes being made

15. Milestones & Timelines

Breakdown the task into major milestone, divide the milestones to further task and subtask, mentioning owners, JIRA/Asana link, due date, reviewer, status
Creating a tabular sheet in excel is preferable here for better understanding
Please paste the link for the tracker with details similar to below here

Sr. No.	Title	Owner	JIRA/Asana link	Due Date	Reviewer	Status
1	Milestone 1 (M1)	Aditi …				
2	Milestone 2 (M1)					

16. Glossary

This section describes all the terms and abbreviations being used specifically in the context of the above feature and heavily being used in this document.

17. Appendix

This section provides the links to all other important documents such as Project Milestone trackers, PRDs (created by Product team), Concept notes, JIRA/Asana project links and any other relevant links.

- Razorpay Product: Concept Note Template <June 2022>
- https://stackoverflow.blog/2020/04/06/a-practical-guide-to-writing-technical-specs/
- https://www.thecloudtutorial.com/technical-specification-document/